

上海财经大学

信息管理与工程学院

随机梯度、Mini-batch 与 Adam

优化理论与算法 大规模机器学习优化

郭加熠

课程导入：为什么这部分很重要

现代机器学习模型通常不是在几百个样本上训练，而是在成千上万、甚至数十亿样本上训练。

传统全梯度

$$\nabla f(w) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w)$$

每一步看全部样本，方向准，但当 n 很大时非常贵。

随机优化

每一步只看一个样本或一小批样本，用便宜但有噪声的方向快速更新。

本节课主线

从 SGD 到 Mini-batch，再到 Momentum、RMSProp 和 Adam：核心都是在“计算成本、噪声、稳定性、调参难度”之间做权衡。

本讲学习目标

- ▶ 理解有限和目标 $f(w) = \frac{1}{n} \sum_i f_i(w)$ 为什么适合随机梯度；
- ▶ 区分 GD、SGD 和 Mini-batch 的更新方式和适用场景；
- ▶ 知道 batch size、epoch、iteration 和学习率调度的含义；
- ▶ 理解动量如何减少随机梯度的抖动和 zig-zag；
- ▶ 理解 AdaGrad、RMSProp、Adam 如何按坐标自适应调整步长；
- ▶ 能结合深度学习、推荐系统、文本模型等案例选择优化器。

有限和目标：大规模学习的标准形式

很多经验风险最小化问题可以写成

$$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w),$$

其中 $f_i(w)$ 是第 i 个样本产生的损失。

例子：线性回归

$$f_i(w) = \frac{1}{2} (x_i^T w - y_i)^2.$$

例子：分类模型

$$f_i(w) = \ell(y_i, h_w(x_i)).$$

例如逻辑回归、神经网络的交叉熵损失。

全梯度需要累加所有样本：

$$\nabla f(w) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w).$$

GD、SGD 与 Mini-batch

GD

$$w^{k+1} = w^k - \alpha_k \frac{1}{n} \sum_{i=1}^n \nabla f_i(w^k).$$

方向准，每步贵。

SGD

$$w^{k+1} = w^k - \alpha_k \nabla f_{i_k}(w^k).$$

每步便宜，噪声大。

Mini-batch

$$w^{k+1} = w^k - \alpha_k \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w^k)$$

兼顾效率和稳定性。

课堂案例：训练图片分类模型

假设我们训练一个图片分类模型，有 $n = 1,000,000$ 张图片。

方法	一次更新用多少图片	课堂理解
GD	1,000,000	每步都看完整数据，方向准但太慢
SGD	1	每步只看一张图，更新快但方向很抖
Mini-batch	32, 64, 128, ...	每步看一小批图，现代深度学习最常用

互动问题

如果 batch size 从 32 增加到 1024，每一步的梯度更准了，但训练一定更好吗？还要考虑哪些因素？

随机梯度为什么可行

随机均匀选取样本 i_k , 有

$$\mathbb{E}_{i_k} [\nabla f_{i_k}(w)] = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w) = \nabla f(w).$$

所以随机梯度是全梯度的无偏估计。

直觉

SGD 每一步方向可能不准, 但平均来看方向是对的。它用更多、更便宜的小步, 替代少量、昂贵的大步。

Mini-batch 梯度同样满足

$$\mathbb{E} \left[\frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w) \right] = \nabla f(w).$$

噪声：SGD 的核心特征

SGD 的方向可以写成

$$g_k = \nabla f_{i_k}(w^k) = \nabla f(w^k) + \xi_k, \quad \mathbb{E}[\xi_k] = 0.$$

好处

- ▶ 每步便宜；
- ▶ 可处理流式或海量数据；
- ▶ 在非凸问题中有时能帮助离开平坦或差的区域。

代价

- ▶ loss 曲线抖动；
- ▶ 不能像 GD 那样平滑下降；
- ▶ 对学习率和 batch size 更敏感。

Epoch、Iteration 与 Batch

随机优化中常见三个计数单位。

概念	含义
iteration	一次参数更新
batch	一次更新使用的样本集合 B_k
batch size	$ B_k $ ，即每次使用多少样本
epoch	全部训练样本大约被使用一遍

例子

若 $n = 60,000$ ，batch size 为 100，则一个 epoch 大约包含 600 次 iterations。

Mini-batch 大小如何影响训练

Mini-batch 梯度为

$$g_{B_k}(w) = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w).$$

batch size 越大，方差越小，但每步计算越贵。

batch size	优点	局限
小	更新频繁，单步快，可能有更强随机性	噪声大，曲线抖动
中等	速度和稳定性折中，适合 GPU 并行	需要结合显存和学习率调参
大	梯度更准，并行效率高	每步贵，可能需要 warmup，泛化未必最好

案例：推荐系统中的 Mini-batch

在推荐系统中，样本可能是“用户--物品--点击/评分”记录：

$$(u_i, item_i, y_i).$$

目标是让模型预测用户是否会点击或喜欢某个物品。

为什么不能全量

每天新增大量交互记录，全量梯度成本高；模型还需要频繁更新以反映用户兴趣变化。

Mini-batch 怎么用

每次抽取一批用户交互记录，计算这一批上的损失和梯度，快速更新 embedding 或模型参数。

这也是为什么 SGD 和 Adam 在推荐、广告、搜索排序中非常常见。

学习率调度

SGD 的学习率非常关键。

- ▶ 固定学习率：实现简单，但后期可能围绕最优点震荡；
- ▶ 衰减学习率：后期更稳定；
- ▶ 分段衰减：训练到某些 epoch 后降低学习率；
- ▶ warmup：训练初期逐渐增大学习率，避免一开始更新过猛；
- ▶ cosine decay：深度学习中常用的平滑衰减策略。

口语版

前期步子要够大，尽快走到好区域；后期步子要变小，避免一直抖动。

SGD 的收敛直觉

对光滑凸问题，在合适学习率下，SGD 常见期望速度为

$$\mathbb{E}[f(\bar{w}^k) - f(w^*)] = \mathcal{O}\left(\frac{1}{\sqrt{k}}\right).$$

如果强凸并使用合适衰减学习率，可以得到更快的期望收敛界。

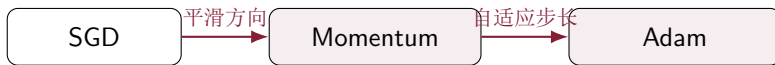
与 GD 的关系

GD 单步方向准确，但每步贵；SGD 单步方向噪声大，但每步便宜。大数据场景中，单位时间内 SGD 往往更有优势。

为什么还需要 Momentum 和 Adam

SGD 好用，但真实训练中常遇到三个问题：

- ▶ 梯度噪声大，方向抖动；
- ▶ 病态问题中 zig-zag 明显；
- ▶ 不同坐标梯度尺度差异大，一个全局学习率难以兼顾所有方向。



SGD with Momentum

动量 SGD 维护一个速度变量:

$$v^{k+1} = \beta v^k + g_k,$$

$$w^{k+1} = w^k - \alpha v^{k+1}.$$

其中 $0 \leq \beta < 1$, g_k 是随机梯度或 mini-batch 梯度。

直觉

如果连续几步梯度方向相似, 动量会累积速度; 如果梯度来回震荡, 动量会相互抵消, 从而减少 zig-zag。

动量的指数平均解释

由递推式

$$\mathbf{v}^{k+1} = \beta \mathbf{v}^k + \mathbf{g}_k$$

展开可得

$$\mathbf{v}^{k+1} = \mathbf{g}_k + \beta \mathbf{g}_{k-1} + \beta^2 \mathbf{g}_{k-2} + \cdots$$

因此 $\mathbf{v}^{k+1}$ 是历史梯度的加权平均，越近的梯度权重越大。

β 小

更看重当前梯度，反应快但平滑弱。

β 大

历史记忆更强，更平滑但可能反应慢或冲过头。

案例：狭长谷地中的 zig-zag

考虑一个病态二次函数：

$$f(x, y) = 50x^2 + y^2.$$

在 x 方向曲率很大，在 y 方向曲率较小。

普通 SGD/GD

如果学习率稍大，沿陡峭方向容易左右摆动；如果学习率很小，沿平坦方向又走得很慢。

Momentum

在持续下降方向上积累速度，在来回摆动方向上相互抵消，因此轨迹更顺。

这就是动量常被用来减少 zig-zag 的原因。

Nesterov 动量

Nesterov 动量的思想是：先根据动量看一眼前方，再计算梯度。

$$\begin{aligned}v^{k+1} &= \beta v^k + \nabla f(w^k - \alpha \beta v^k), \\w^{k+1} &= w^k - \alpha v^{k+1}.\end{aligned}$$

口语版

普通动量像“先看当前坡度再冲”；Nesterov 像“先按惯性预判一下会到哪里，再看那里的坡度”。

AdaGrad: 按坐标缩放步长

AdaGrad 记录每个坐标历史梯度平方和:

$$r_k = r_{k-1} + g_k \odot g_k.$$

更新为

$$w^{k+1} = w^k - \alpha \frac{g_k}{\sqrt{r_k} + \epsilon}.$$

直觉

某个坐标过去梯度经常很大, 就减小该坐标步长; 很少被更新的坐标保持相对较大的步长。

AdaGrad 对稀疏特征很有用, 但 r_k 单调增大, 后期步长可能过小。

案例：文本模型中的稀疏特征

在文本分类或搜索排序中，特征可能是词语、短语或类别 ID。很多特征非常稀疏：

“通货膨胀”出现频率高， “近端梯度法”出现频率低。

频繁特征

历史梯度平方和大，AdaGrad 会自动减小步长，避免更新过猛。

稀有特征

历史梯度平方和小，仍保留较大学习率，更容易被学习到。

RMSProp: 遗忘很久以前的梯度

RMSProp 使用指数平均:

$$r_k = \beta r_{k-1} + (1 - \beta)(g_k \odot g_k).$$

更新为

$$w^{k+1} = w^k - \alpha \frac{g_k}{\sqrt{r_k} + \epsilon}.$$

与 AdaGrad 的差别

AdaGrad 累加所有历史平方梯度; RMSProp 对历史做衰减, 更关注最近一段时间的梯度尺度, 因此后期步长不至于无限变小。

Adam: 动量 + 自适应步长

Adam 同时维护一阶矩和二阶矩:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k,$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2)(g_k \odot g_k).$$

偏差修正:

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}.$$

更新:

$$w^{k+1} = w^k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}.$$

Adam 怎么理解

一阶矩 m_k

类似 momentum，平滑梯度方向，减少噪声。

二阶矩 v_k

估计每个坐标的梯度尺度，用来调节每个坐标的步长。

Adam 的更新可以口语化理解为：

平滑后的方向 / 每个坐标的近期梯度尺度.

常见默认值

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}.$$

为什么 **Adam** 这么流行

- ▶ 对不同坐标尺度比较鲁棒；
- ▶ 对稀疏梯度、深层网络、embedding 参数通常较友好；
- ▶ 初期 loss 下降快，容易得到可用模型；
- ▶ 在 PyTorch、TensorFlow 等框架中默认支持，工程使用方便。

典型应用

大语言模型预训练、图像分类、推荐系统、广告点击率预测、强化学习中的神经网络训练等。

Adam 不是万能的

Adam 在深度学习中非常常用，但需要注意：

- ▶ 对学习率仍然敏感；
- ▶ 训练 loss 快不一定代表泛化最好；
- ▶ 在某些任务上 SGD + momentum 的最终泛化表现可能更好；
- ▶ 权重衰减需要小心处理，实践中常用 AdamW；
- ▶ 理论收敛分析比 GD、SGD 更复杂。

实践建议

Adam 常适合快速得到可用模型；若追求最终泛化，可尝试学习率衰减、AdamW，或后期切换到 SGD + momentum。

AdamW: 深度学习中的常见变体

很多深度学习模型会使用权重衰减来控制模型复杂度。AdamW 的思想是把权重衰减从 Adam 的自适应梯度更新中解耦出来。

$$w^{k+1} = w^k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon} - \alpha \lambda w^k.$$

Adam

自适应梯度更新，适合快速训练。

AdamW

更清晰地加入权重衰减，是 Transformer、视觉模型中常见选择。

优化器选择：课堂决策表

场景	常见选择
小规模、光滑、要求高精度	GD、Newton、BFGS、L-BFGS
大规模凸问题或线性模型	SGD、Mini-batch SGD、AdaGrad
深度神经网络初期训练	Adam、AdamW、RMSProp
图像分类追求最终泛化	SGD + momentum，配合学习率衰减
稀疏特征、推荐和文本 embedding	AdaGrad、Adam、AdamW

选择原则

先看数据规模和梯度成本，再看噪声、坐标尺度、泛化需求和工程约束。

常见误区

误区	正确理解
Adam 一定比 SGD 好	Adam 初期常快，但最终泛化未必总优
batch size 越大越好	大 batch 梯度准，但每步贵，也可能需要更复杂学习率策略
loss 抖动说明算法失败	SGD 抖动是正常现象，需要结合趋势、epoch 和验证集判断
默认学习率总能用	优化器默认参数常能启动训练，但好结果仍需调参

本讲总结

- ▶ 大规模机器学习目标常是有限和 $f(w) = \frac{1}{n} \sum_i f_i(w)$ 。
- ▶ GD 每步使用所有样本，方向准但代价高；SGD 和 Mini-batch 用噪声方向换取低成本更新。
- ▶ batch size、epoch、iteration 和学习率调度是理解训练过程的基本语言。
- ▶ Momentum 用历史梯度平滑方向，减少 zig-zag。
- ▶ AdaGrad、RMSProp 和 Adam 按坐标调整步长，适合梯度尺度差异大或稀疏特征场景。
- ▶ Adam/AdamW 很流行，但不是万能；优化器选择要结合任务、数据规模和泛化需求。

课后思考

为什么很多深度学习训练先用 Adam 快速下降，而一些任务后期又会尝试 SGD + momentum？